

# Bocconi

## COMPUTER SCIENCE

code 30424 a.y. 2024-2025

### **Lesson 15**

Introduction to Control Flow Statements (iterative structures)



Universit   
Bocconi  
MILANO

# Objectives of the lesson

- Understand what a repetition structure is
- Learn how to create different types of iterative constructs

# Repetition structures

- A repetition structure is defined by **iterative constructs** that have the ability to execute repeatedly the same block of statements
- Iteration is an essential feature of programming languages, which is implemented by means of constructs called **loops**
- In Python loops are grouped into two categories: loops **controlled by a condition** and loops **controlled by a counter**
- Like many other programming languages, Python's main iterative constructs are **while** and **for** loops

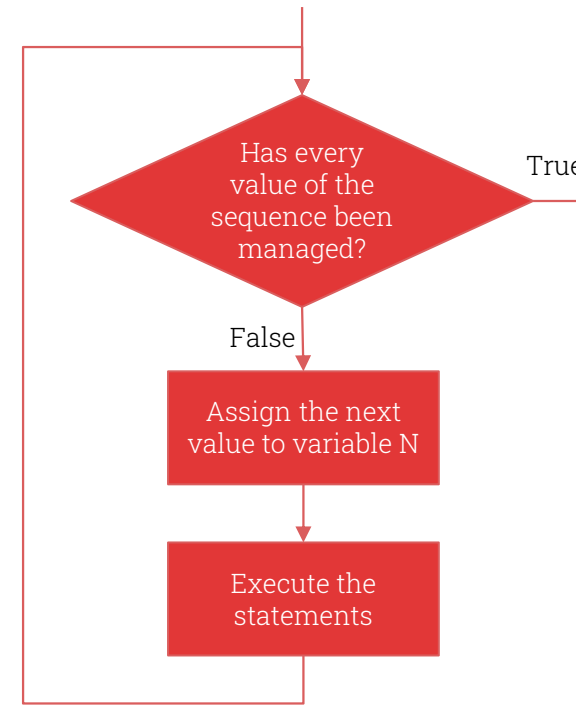
# Loops controlled by a counter

**for** statement enables to create a loop where a block of statements is repeatedly executed **for each value** of a sequence

The syntax is:

```
for N in [value 1, value 2, value 3]:  
    statement  
    statement  
    statement  
    ...  
[rest of the script]
```

Indentation



# Example of for loop

Example:

```
for num in [1, 2, 3, 4, 5]:  
    print(num)
```

Output:

```
1  
2  
3  
4  
5  
>>>
```

Variations:

1. Modify the example by using the `range` function
2. Modify the example by using `range(5)`
3. Modify the example to show numbers **from 5 to 1**, using the `range` function

# Exercise on for loop

Create a program to ask the user to enter an integer number, in order to print the multiplication table of the given number (i.e. all its multiples, from **n** to **n\*10**).

Result:

```
Enter an integer number: 8
```

```
8 x 1 = 8
```

```
8 x 2 = 16
```

```
8 x 3 = 24
```

```
8 x 4 = 32
```

```
8 x 5 = 40
```

```
8 x 6 = 48
```

```
8 x 7 = 56
```

```
8 x 8 = 64
```

```
8 x 9 = 72
```

```
8 x 10 = 80
```

```
>>>
```



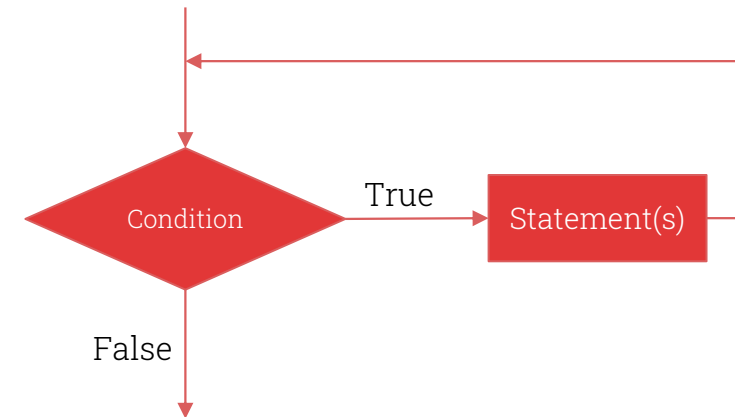
# Loops controlled by a condition

**while** statement enables to create a loop where a block of statements is repeatedly executed **as long as the condition remains true**

The syntax is:

```
while condition:  
    statement  
    statement  
    statement  
    ...  
[rest of the script]
```

Indentation



# Example of while loop

Example:

```
x = 1
while x <= 10:
    print(x)
    x = x+1
```

The  
variable  
must be  
initialized

Output:

```
1
2
3
4
5
6
7
8
9
10
```

You can  
also write  
x+=1

Variations:

1. Modify the example to print «bye bye» after the last number
2. Modify the example to print the same list of numbers in a reversed order (from 10 to 1)
3. Modify the example to print a sequence of numbers based on the inputs given by the user (e.g. all even numbers from 8 to 24)



# Example of while loop (2)

```
keep_going = 'y'

while keep_going == 'y':

    ID_num = input('\nEnter the student ID number: ')
    last_name = input('Enter the last name: ')
    name = input('Enter the name: ')
    print('\nWelcome', name, last_name, '('+ID_num+')')

    keep_going = input('Do you want to enter data for another student? (y/n) ')

```

```
Output: Enter the student ID number: 3201224
        Enter the last name: Smith
        Enter the name: John

        Welcome John Smith (3201224)
        Do you want to enter data for another student? (y/n) y

        Enter the student ID number: 3215472
        Enter the last name: Doe
        Enter the name: Jane

        Welcome Jane Doe (3215472)
        Do you want to enter data for another student? (y/n) n
        >>> |

```



# The break and continue statements

- In some cases, **for** and **while** loops must be interrupted when a specific condition occurs
- In other cases, it is necessary to move on to the next iteration when specific conditions occur
- To manage these cases, the **break** and **continue** statements can be used respectively

# The break and continue statements

Examples:

```
# Prints odd numbers between 1 and 50

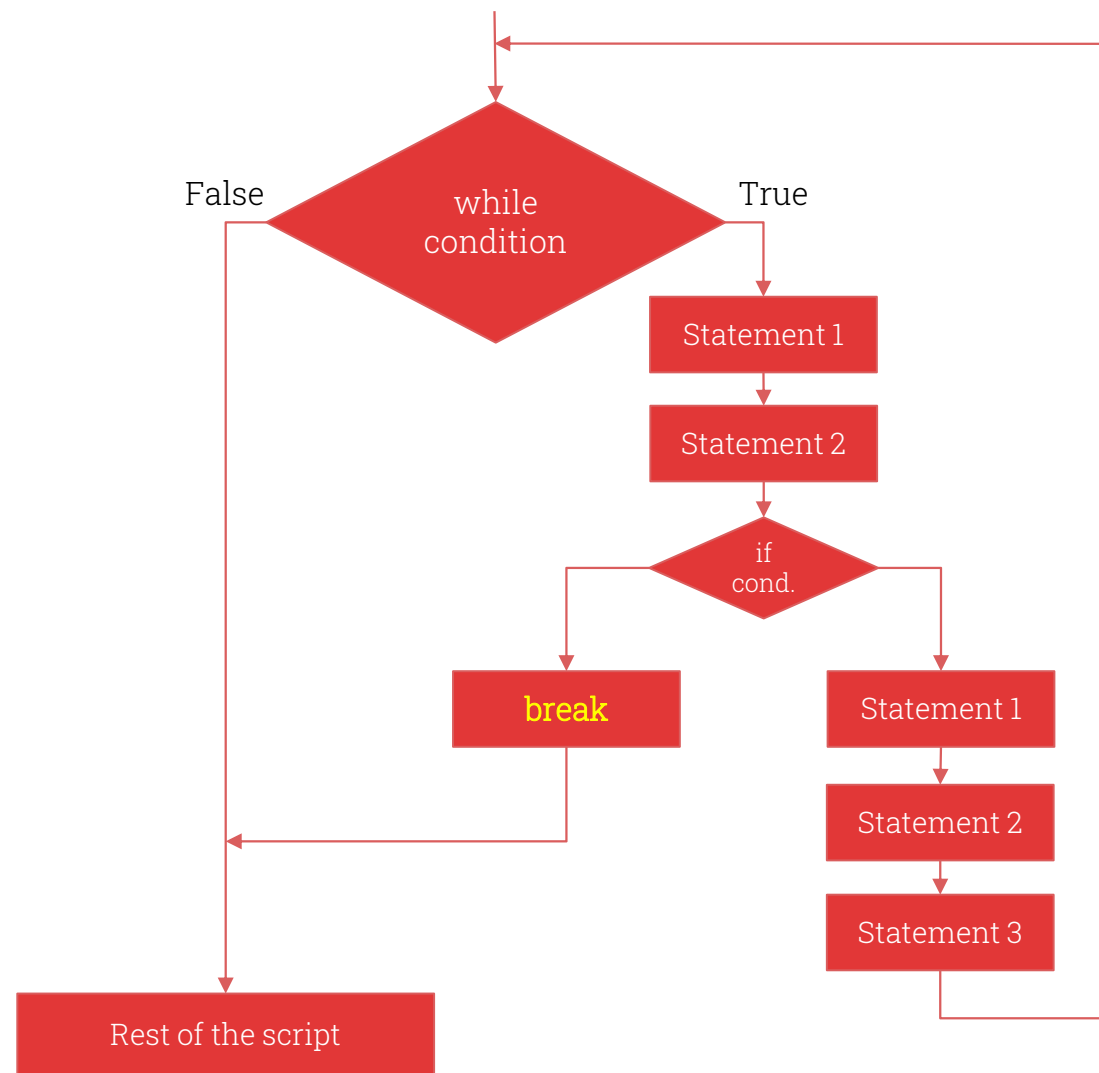
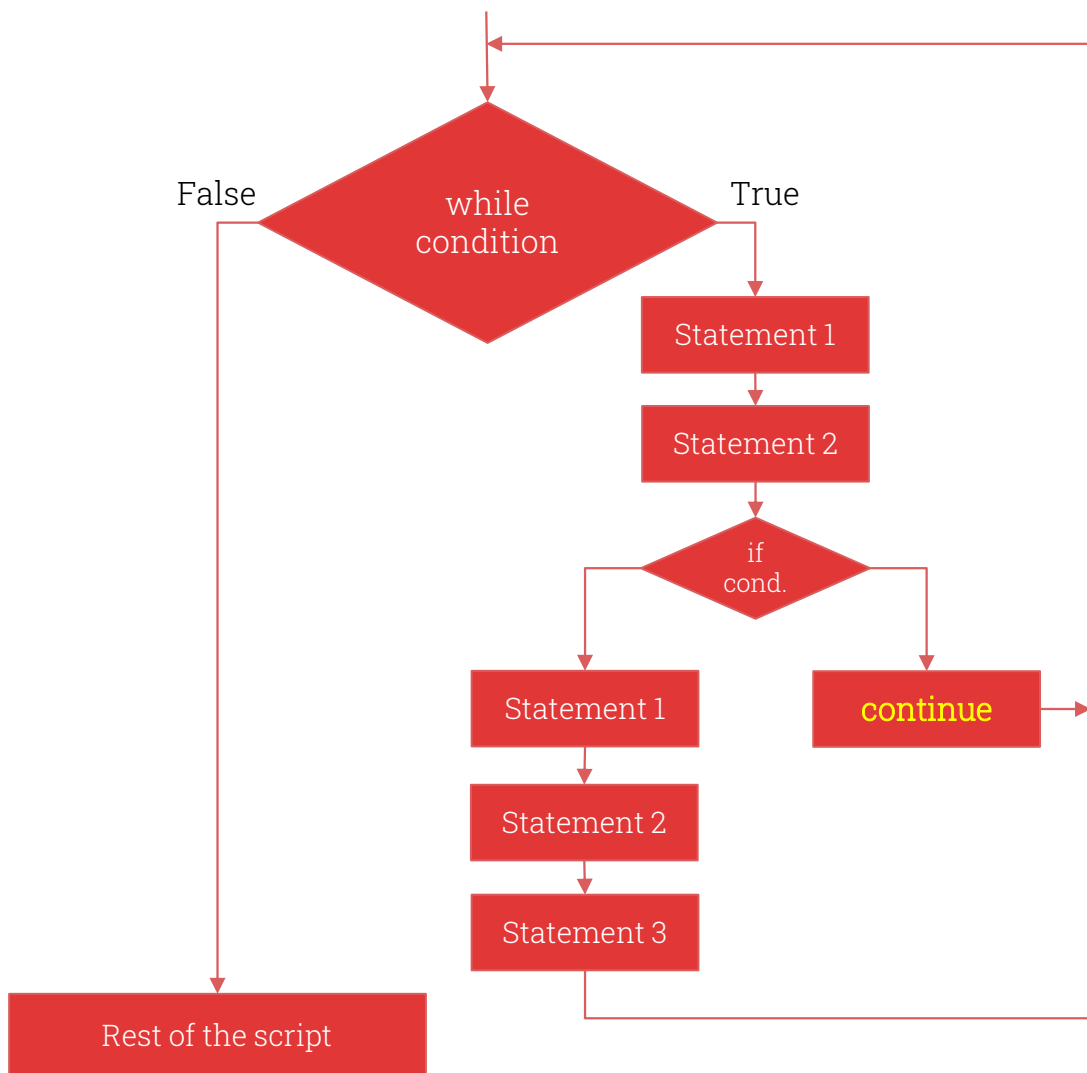
for number in range(1, 51):
    if number % 2 == 0:
        continue
    print("Odd number found:", number)
```

```
'''
Prints all exponents of a given base raised to powers ranging
from 0 to 10, up to the highest value not exceeding 1000
'''

base = int(input("Please, enter a number between 2 and 10: "))
for n in range(11):
    num = base**n
    if num > 1000:
        print("Any other value is greater than 1000.")
        break
    print(base, "**", n, "=", num)
```



# The break and continue statements



# Exercise 1

Create a new file named **coins.py** and write a program to simulate a coin toss with the aim of verifying empirically that by throwing a coin (not rigged) you have an equal chance of getting "heads" or "tails" (50% heads - 50% tails). The program must:

- ask the user to enter the number of coin tosses to be simulated
- simulate the coin toss for the given number of times and store the outcome in two variables (heads and tails). **Note:** use the **randint** function of the random module to assign an integer value to "heads" and another integer value to "tails".
- Show the result of the simulation using the following message (to be printed on two lines):

*Heads tosses = XX*

*Tails tosses = XX*



## Exercise 2

Create a new Python file named **divisors.py** that calculates all the divisors of a number entered by the user. In particular, the program must:

- ask the user for an integer
- show on screen each divisor of the given number
- show on screen the total number of divisors found

# Tools to learn with today's lecture

- Iterative constructs: **while**, **for**
- **break**, **continue** statements

# Book references

Understanding Python:

Chapter 6

Python workbook:

Exercises of Unit 3